
Travaux Pratiques

TD/TP: Entraînement Distribué d'un Modèle BERT — Parallélisme de Données Multi-GPU avec PyTorch DDP

Réalisé par :

BOUYOUS HOUSSAME
CHBAB BRAHIM
BENJELLOUN ABDERRAHMANE

Encadrants :

Pr. CHAIMAA LAMHARMECH
Pr. RAWANE AZEROUAL
Pr. YASSIR AHRAR

Objectifs

Ce TP a pour objectif d'introduire la programmation parallèle distribuée appliquée au Deep Learning, à travers le fine-tuning d'un modèle BERT pour la classification de sentiments (dataset IMDB). À la fin de cette séance, l'étudiant sera capable de :

- Distinguer l'entraînement sur un seul GPU et l'entraînement distribué multi-GPU
- Utiliser PyTorch Lightning et DistributedDataParallel (DDP) avec les paramètres devices, strategy et precision
- Mesurer et analyser un speedup réel sur une charge de travail Deep Learning suffisamment grande pour être significative
- Comprendre le rôle de la synchronisation des gradients (AllReduce / NCCL)
- Appliquer la loi d'Amdahl pour expliquer les performances d'un entraînement multi-GPU

Prérequis — Environnement

Ce TP nécessite un environnement disposant d'au moins 2 GPU (idéalement 2× NVIDIA T4, 16 Go de VRAM chacun).

Sur Kaggle Notebook (recommandé) :

- Créer un nouveau notebook, puis dans Settings → Accelerator, choisir GPU T4 x2
- Vérifier que Internet est activé (nécessaire pour télécharger le dataset IMDB et les poids BERT)

Sur Google Colab :

- Colab ne fournit qu'un seul GPU par session gratuite : l'Expérience B (multi-GPU) nécessite Colab Pro+ ou un environnement disposant de plusieurs GPU

Installation des bibliothèques (exécutée automatiquement dans les notebooks) :

```
pip install "transformers>=4.41.0" "datasets>=2.20.0" "pytorch-lightning==2.2.4"
"torchmetrics==1.4.0"
```

Fichiers fournis

L'archive du TP contient les fichiers suivants :

- single-gpu-experiment.ipynb — Expérience A : baseline sur 1 GPU + démonstration du mur mémoire (OOM), avec timing par epoch visible en direct
- multi-gpu-experiment.ipynb — Expérience B : entraînement distribué DDP sur 2 GPU, même timing par epoch, même taille de dataset
- train_ddp.py — généré automatiquement par le notebook multi-GPU, lancé via torchrun

Partie I — Prise en main de l'environnement multi-GPU

Exercice 1 : Vérification de l'environnement

Compilation et exécution (notebook single-gpu-experiment.ipynb, Section 1) :

```
python train_ddp.py --num-gpus 1 # équivalent à : ./ex1 sur un seul «thread» GPU
```

1.1 — Affichage des GPU disponibles

Exécutez la cellule STEP 2 qui interroge `torch.cuda.device_count()` et affiche le nom et la mémoire de chaque GPU visible.

Question : Les GPU s'affichent-ils tous avec la même quantité de mémoire libre ? Un GPU peut-il être « plus rapide » qu'un autre alors qu'ils sont du même modèle ? Expliquez ce qui pourrait causer un déséquilibre entre GPU dans un cluster réel.

1.2 — Chargement et découpage reproductible du dataset

Le dataset IMDb (25 000 critiques d'entraînement, 25 000 de test) est téléchargé puis tokenisé avec `BertTokenizer` (`max_length=128`). On utilise volontairement une grande partie du dataset — `TRAIN_SIZE=20000`, `VAL_SIZE=5000` — afin que le temps de calcul GPU domine largement les coûts fixes (téléchargement, tokenisation, démarrage de `torchrun`) et que le speedup mesuré entre 1 et 2 GPU soit réellement significatif. Un générateur à seed fixe (`torch.Generator().manual_seed(42)`) est utilisé pour le split train/val :

```
g = torch.Generator().manual_seed(42)train_subset, _ = random_split(tokenized['train'], [TRAIN_SIZE, ...], generator=g)
```

Question : Pourquoi est-il indispensable d'utiliser le même seed — et la même valeur de `TRAIN_SIZE/VAL_SIZE` — dans `single-gpu-experiment.ipynb` et dans `train_ddp.py` pour que la comparaison de speedup soit valide ? Que se passerait-il si chaque expérience utilisait un split aléatoire différent, ou une taille de dataset différente ?

Partie II — Entraînement du modèle BERT

Exercice 2 — Expérience A : baseline sur un seul GPU

Compilation et exécution (notebook single-gpu-experiment.ipynb, Section 4) :

```
# Cellule STEP 7 :trainer_single = pl.Trainer(accelerator="gpu", devices=1,
strategy="auto", enable_progress_bar=False,
callbacks=[EpochTimer()], ...)trainer_single.fit(model_single, train_loader, val_loader)
```

Le modèle `BERTSentimentClassifier` (110 millions de paramètres) est entraîné pendant 2 époques sur 20 000 exemples, avec un `batch_size=32` et la précision mixte 16-mixed (équivalent GPU de la vectorisation SIMD : elle réduit le volume de données à déplacer et accélère le calcul sans changer le résultat de manière significative).

Le callback EpochTimer affiche le temps de chaque epoch dès qu'elle se termine (au lieu d'un seul chiffre final caché derrière la barre de progression) : vous devez voir une ligne « 🕒 Epoch k/2 terminée en ...s » s'afficher pendant l'exécution, pas seulement à la fin.

Question 2.1 : Mesurez le temps d'entraînement (STEP 7), epoch par epoch, et complétez le tableau ci-dessous. Exécutez ensuite le même entraînement avec 2 GPU (notebook multi-gpu-experiment.ipynb) et calculez le speedup.

GPU utilisés	Batch global	Temps (s)	Speedup	Gain (%)
1	32		1.00	0 %
2 (DDP)	256 (128×2)			

Question 2.2 : Pourquoi peut-on dire que l'entraînement par mini-batch est « embarrassingly parallel » au niveau des données (data parallelism)? Quelle opération, en revanche, n'est PAS embarrassingly parallel et nécessite une synchronisation entre GPU ?

Exercice 3 — Le mur mémoire : Out-Of-Memory sur un seul GPU

Compilation et exécution (notebook single-gpu-experiment.ipynb, Section 4, STEP 8) :

```
large_batch_loader = DataLoader(train_subset, batch_size=1024,
...)trainer_oom.fit(model_oom, large_batch_loader, val_loader) # → RuntimeError: CUDA out of memory
```

Un seul GPU T4 (16 Go de VRAM) ne peut pas contenir en mémoire les activations, gradients et états de l'optimiseur AdamW pour un batch de 1024 séquences BERT en même temps (cette limite dépend de la taille du batch, pas de la taille totale du dataset — augmenter TRAIN_SIZE ne change donc rien à ce test).

Question 3.1 : Confirmez l'erreur OOM en exécutant la cellule STEP 8. Quelles sont les trois composantes qui occupent la mémoire GPU pendant l'entraînement (en dehors des poids du modèle lui-même) ?

Question 3.2 : Le notebook multi-gpu-experiment.ipynb répartit un batch global de 256 en 128 par GPU sur 2 GPU, ce qui est très inférieur au batch de 1024 qui a fait planter l'Exercice 3.1. Pourquoi peut-on quand même affirmer que le principe démontré ici (répartir un gros batch entre plusieurs GPU pour réduire la charge mémoire par GPU) s'applique de la même façon, même si les deux exercices n'utilisent pas la même taille de batch ?

Exercice 4 — Expérience B : entraînement distribué avec DDP

Compilation et exécution (notebook multi-gpu-experiment.ipynb, Section 4) :

```
torchrun --standalone --nproc_per_node=2 train_ddp.py \    --batch-size-per-gpu 128
--epochs 2 --train-size 20000 --val-size 5000 --num-gpus 2
```

torchrun lance deux processus système indépendants (un par GPU). PyTorch Lightning configure automatiquement un DistributedSampler qui donne à chaque GPU une portion disjointe du jeu d'entraînement. Le callback EpochTimer n'imprime qu'une seule ligne par epoch (filtrée sur trainer.is_global_zero) même si les 2 processus s'exécutent en parallèle.

Question 4.1 : Ouvrez train_ddp.py (généré par la cellule STEP 6). Combien de processus torchrun crée-t-il si --nproc_per_node=2 ? Quelles variables d'environnement (RANK, LOCAL_RANK, WORLD_SIZE) chaque processus reçoit-il, et à quoi servent-elles ?

Question 4.2 : Pourquoi le notebook affiche-t-il les logs de progression uniquement pour le processus is_main_process / trainer.is_global_zero (rank == 0) ? Que se passerait-il si les 2 processus écrivaient tous les deux dans le même fichier results_multi_gpu.json en même temps ?

Exercice 5 — Synchronisation des gradients (AllReduce)

Dans `BERTSentimentClassifier.training_step()`, chaque appel à `self.log(..., sync_dist=True)` déclenche une opération `AllReduce` via `NCCL` qui moyenne la métrique entre tous les GPU avant de l'afficher

```
self.log("train_loss", loss, prog_bar=True, sync_dist=True)
```

Question 5.1 : Que se passerait-il si vous retiriez `sync_dist=True` ? Chaque processus GPU afficherait-il la même valeur de `train_loss`, ou une valeur propre à son propre lot de données ?

Question 5.2 : Les GRADIENTS eux-mêmes (utilisés pour la mise à jour des poids) sont-ils synchronisés de la même manière que les métriques de logging ? Que se passerait-il pour la cohérence du modèle si les gradients n'étaient PAS synchronisés entre GPU à chaque pas d'entraînement ?

Partie III — Analyse scientifique

Exercice 6 — Loi d'Amdahl appliquée au Deep Learning distribué

La loi d'Amdahl prédit le speedup maximal théorique d'un programme partiellement parallélisé :

$$S(N) = 1 / (s + (1 - s) / N)$$

où s est la fraction séquentielle du programme (chargement des données, tokenization, initialisation du modèle, sauvegarde du checkpoint) et N le nombre de GPU.

Question 6.1 : À partir de vos mesures de l'Exercice 2 (1 GPU et 2 GPU), complétez le tableau suivant en supposant $s = 0.10$ (10 % du temps total est intrinsèquement séquentiel : téléchargement du dataset, tokenization, chargement des poids pré-entraînés).

GPU (N)	Temps mesuré (s)	Speedup mesuré	Speedup Amdahl (s=0.10)	Écart
1		1.00	1.00	0 %
2			1.82	

Question 6.2 : À partir de vos mesures réelles (et non de l'hypothèse $s = 0.10$), estimez la fraction séquentielle s de votre pipeline d'entraînement en résolvant $S(2) = 1 / (s + (1 - s)/2)$ pour la valeur de speedup mesurée.

Question 6.3 : Quel serait le speedup maximum théorique avec un nombre infini de GPU ? Rappel : $S_{\max} = 1 / s$. Ce maximum est-il réaliste pour un cluster de calcul réel ?

Question 6.4 : Citez trois raisons, spécifiques au Deep Learning distribué, qui expliquent l'écart entre le speedup mesuré et le speedup théorique d'Amdahl. Pensez notamment à : la bande passante d'interconnexion GPU-GPU (PCIe/NVLink) utilisée par l'`AllReduce`, le coût fixe de démarrage des processus `torchrun`, et le chargement des données (`DataLoader` et tokenisation, souvent limités par le CPU et potentiellement dupliqués entre processus — cf. l'indice ci-dessus).

Pour aller plus loin (optionnel)

- Passez `batch-size-per-gpu` à 256 et observez si l'`OOM` réapparaît même en DDP — quelle solution proposeriez-vous (gradient accumulation, ZeRO, etc.) ?
- Comparez le speedup obtenu avec 2 GPU à celui d'un entraînement où seul le batch size est augmenté sur 1 GPU sans parallélisme (effet sur la vitesse de convergence vs. la vitesse brute)
- Étudiez l'effet de la précision (32 vs 16-mixed vs bf16) sur le temps d'entraînement et la mémoire utilisée
- Comparez, à charge par GPU égale (batch=128 sur 1 GPU vs 128×2 en DDP), le speedup « pur » du parallélisme sans l'effet confondu de l'augmentation du batch global
- Relancez l'Exercice 4 avec `--batch-size-per-gpu 512` (soit un batch global de 1024, identique à celui qui a fait planter l'Exercice 3.1) : confirmez que ce même batch de 1024, qui ne tenait pas sur 1 GPU, s'exécute sans erreur une fois réparti en 512+512 sur 2 GPU